

**N****Nimbus Bank**

Guia do Frontend

Como cada parte da interface funciona

Aplicacao React + Vite de um banco digital, com autenticao JWT, contas, Pix, transferencias, extrato e notificacoes.

1. Visao geral

O frontend e uma **Single Page Application (SPA)** escrita em React. Ele conversa com a API Spring Boot por HTTP (Axios) e nunca acessa o banco diretamente - tudo passa pela API, protegida por token JWT.

Tecnologias principais

- **React 18 + Vite** - biblioteca de UI e ferramenta de build/dev server (porta 3000).
- **React Router** - navegacao entre paginas sem recarregar (rotas e guardas).
- **Axios** - cliente HTTP; injeta o token e renova a sessao automaticamente.
- **Recharts** - grafico de evolucao no painel.
- **CSS proprio** - tema escuro estilo banco digital, sem framework de UI.

Estrutura de pastas (src/)

```
src/  
|- api/           comunicacao com a API (axios + endpoints)  
|- auth/          protecao de rotas (ProtectedRoute)  
|- context/       estado global (sessao JWT e toasts)  
|- components/    pecas reutilizaveis (layout, modal, inputs, icones)  
|- pages/         cada tela do app (login, contas, pix, perfil...)  
|- styles/        CSS global, layout e telas de login  
|- utils/         funcoes utilitarias (formatar moeda e datas)
```

2. Como funciona a autenticacao (JWT)

E o coracao da seguranca no frontend. O fluxo e:

- **1. Login/Cadastro** - a tela envia email/senha; a API responde com um *access token* (curto, 15 min) e um *refresh token* (7 dias).
- **2. Armazenamento** - os tokens e os dados do usuario ficam no `localStorage`, gerenciados pelo **AuthContext**.
- **3. Em cada requisicao** - o Axios injeta o cabecalho `Authorization: Bearer <token>` automaticamente.
- **4. Renovacao automatica** - se a API responde 401 (token expirado), o Axios chama `/auth/refresh`, pega um token novo e refaz a requisicao - sem o usuario perceber.
- **5. Protecao de telas** - o **ProtectedRoute** bloqueia paginas internas para quem nao esta logado.
- **6. Logout** - revoga o refresh token na API e limpa o `localStorage`.

Em resumo: o usuario faz login uma vez e a sessao se mantem sozinha, de forma segura, ate o logout ou a expiracao do refresh token.

3. Pasta api/ - conversa com o servidor

client.js

Cria a instancia do Axios e os **interceptors**: um adiciona o token em toda requisicao; outro trata o 401 renovando o token (refresh) e refazendo a chamada. Tambem expoe `apiError()`, que extrai a mensagem de erro amigavel da API.

services.js

Lista os endpoints por area: `authApi`, `contaApi`, `clienteApi`, `pixApi` e `notificacaoApi`. Tem `baixarBlob()` (download de PDF/CSV) e desembrulha listas paginadas (`Page`) em arrays.

4. Pasta context/ - estado global

AuthContext.jsx

Guarda quem esta logado e os tokens. Oferece `login`, `register`, `logout`, `isAuthenticated` e `isAdmin`. No carregamento valida o token chamando `/auth/me`.

ToastContext.jsx

Notificacoes rapidas no canto da tela (sucesso/erro/info). Qualquer pagina chama `toast.success(...)` ou `toast.error(...)`.

5. Pasta auth/ - protecao de rotas

ProtectedRoute.jsx

Envolve as paginas internas. Sem sessao valida, redireciona ao login; enquanto valida o token, mostra um carregando. E o porteiro do app.

6. Pasta components/ - pecas reutilizaveis

AppLayout.jsx	O esqueleto das telas logadas: menu lateral (Visao geral, Contas, Transferir, Pix e - so para ADMIN - Clientes), barra superior com o sino de notificacoes e o avatar (que abre o Perfil), e a area onde cada pagina e renderizada.
NotificationBell.jsx	O sino da barra superior. Mostra o contador de nao lidas (consulta a API a cada 30s) e abre um painel com as notificacoes; permite marcar como lidas.
MoneyInput.jsx	Campo de valor com mascara de moeda . O usuario digita so numeros e o componente formata como R\$ enquanto digita (trabalha em centavos), evitando valores invalidos.
Modal.jsx	Janela sobreposta reutilizavel (deposito, saque, nova conta, nova chave Pix). Fecha com Esc ou clique fora.
Loader.jsx	Indicador de carregamento (spinner), pagina inteira ou pequeno.
Icons.jsx	Conjunto de icones em SVG (carteira, Pix, sino...), sem dependencia externa.
Common.jsx	Pecas compartilhadas: selos de status da conta, estado vazio e o estilo visual de cada tipo de transacao.
AuthBrand.jsx	Painel lateral de marca nas telas de login e cadastro.

7. Pasta pages/ - as telas

Login.jsx	Entrada do usuario (email e senha). Em erro, mostra a mensagem da API.
Register.jsx	Cadastro: nome, email, CPF, celular e senha. Cria login + perfil de cliente.
Dashboard.jsx	Painel inicial: patrimonio consolidado, grafico de evolucao, atividade recente e atalhos para as contas.
Contas.jsx	Lista as contas (com filtros por status) e permite criar uma nova conta.
ContaDetalhe.jsx	Tela de uma conta: saldo, operacoes (depositar, sacar, transferir, ajustar limite, bloquear/ativar, encerrar), extrato e exportacao em PDF/CSV.
Transferir.jsx	Transferencia entre contas pelo ID do destino, com valor mascarado e mensagem.
Pix.jsx	Area Pix com duas abas: Transferir (por chave) e Minhas chaves (cadastrar/excluir chaves de email, CPF, celular ou aleatoria).
Perfil.jsx	Mostra e edita os dados do titular (nome, email, celular) via /clientes/me. Aberto ao clicar no avatar.
Clientes.jsx	Gestao de clientes - visivel apenas para ADMIN (lista paginada).

8. Pastas styles/ e utils/

styles/global.css	Variaveis de cor do tema, botoes, inputs, selos e utilitarios base.
styles/layout.css	Layout logado: menu lateral, barra superior, cards, tabelas, modal, notificacoes e responsividade.
styles/auth.css	Estilos das telas de login e cadastro.
utils/format.js	Formatacao: moeda em R\$, datas, iniciais do avatar e mascara do numero da conta.

9. Fluxos principais (passo a passo)

Login

Login.jsx → AuthContext.login() → authApi.login() → API devolve tokens → salvos no localStorage → vai ao Dashboard

Transferencia

Transferir.jsx → MoneyInput (valor) → contaApi.transferir() → API valida saldo/dono → toast de sucesso → notificacao nos dois lados

Pix por chave

Pix.jsx (aba Transferir) → pixApi.transferir() → API resolve a chave -> conta destino → transfere → notificacao para quem recebe

Token expirado

requisicao falha 401 → interceptor do client.js → /auth/refresh → novo token → requisicao refeita automaticamente

Exportar extrato

ContaDetalhe.jsx (PDF/CSV) → contaApi.extratoPdf/csv (blob) → baixarBlob() → download no navegador

Como rodar

```
cd frontend
cp .env.example .env
npm install
npm run dev # http://localhost:3000
```

E necessario o backend Spring Boot em http://localhost:8080. Login de demonstracao: admin@bank.com / admin123.